Trabajo Práctico N° 3 — Permisos móviles: Android, iOS y Expo

Alumno/a:		Fecha de entrega:	

Consignas generales

Objetivo: aplicar la teoría de permisos móviles para que una app hecha con React Native y Expo (SDK 57) pida acceso a la cámara, la ubicación, las fotos y las notificaciones **en el momento justo**, lea la respuesta del sistema y tenga un camino alternativo para cada caso, tanto en Android como en iOS.

Modalidad: las Partes A a F son de preguntas y ejercicios cortos; respondelas en un archivo `RESPUESTAS.md` dentro del repositorio del proyecto. La Parte G es el desarrollo del sistema. Todo se entrega en un único repositorio.

Material de consulta: la presentación de clase “Permisos móviles” y la guía oficial de Expo (docs.expo.dev/guides/permissions).

Parte A · ¿Qué es un permiso y por qué existe?

A1. Conceptos

Respondé con tus palabras:

a) ¿Qué es un permiso? ¿Qué papel cumplen **la app**, **el sistema operativo** y **la persona usuaria**?

b) ¿Qué es la **caja de arena** (sandbox) y qué relación tiene con los permisos? ¿Qué puede hacer una app sin ningún permiso?

c) Nombrá las cuatro razones por las que existen los permisos y dá un ejemplo concreto de cada una.

d) En la analogía del portero del edificio, ¿quién es la visita, quién el portero y quién el dueño del departamento? ¿Qué significa que “el portero se acuerda de la respuesta”?

A2. Declarar y solicitar

Todo permiso pasa por dos momentos. Completá la tabla.

	Declarar (al compilar)	Solicitar (al ejecutar)
¿Qué se hace?		
¿Dónde / cómo en Android?		
¿Dónde / cómo en iOS?		

	Declarar (al compilar)	Solicitar (al ejecutar)
¿Dónde / cómo en Expo?		
¿Qué pasa si se omite?		

A3. Verdadero o falso

Indicá V o F y justificá las falsas.

a) El diálogo de permiso lo dibuja tu app con componentes de React Native.

b) En Expo, la fuente de verdad de los permisos es `app.json` (o `app.config.ts`), no el `AndroidManifest.xml`.

c) Agregar un permiso nuevo en el `Info.plist` se puede publicar con una actualización OTA.

d) Si tu código pide un permiso que no está en el manifiesto, Android igual muestra el diálogo.

e) El paquete `expo-permissions` es la forma recomendada de pedir permisos en SDK 57.

f) En iOS, las notificaciones no llevan texto de propósito en el `Info.plist`.

g) Con acceso limitado a fotos, `status` vale `'granted'` y el matiz está en `accessPrivileges`.

h) Desinstalar y reinstalar la app es la forma confiable de volver a “sin decidir” en las dos plataformas.

A4. Los estados de un permiso

a) Dibujá (o describí) el diagrama de estados de un permiso en Android y en iOS. ¿Cuál es la diferencia central entre las dos plataformas?

b) Completá qué devuelve Expo en cada estado:

Estado	status	canAskAgain	¿Puede aparecer el diálogo?
Sin decidir			
Concedido			
Denegado (se puede volver a pedir)			
Bloqueado			

Parte B · Android: tipos de permisos y manifiesto

B1. Clasificá los permisos

Indicá el tipo (de instalación / de tiempo de ejecución / especial) y cómo los obtiene la app.

Permiso	Tipo	¿Cómo se obtiene?
`INTERNET`		
`CAMERA`		
`VIBRATE`		
`SYSTEM_ALERT_WINDOW`		
`POST_NOTIFICATIONS`		
`ACCESS_NETWORK_STATE`		
`READ_CONTACTS`		
`SCHEDULE_EXACT_ALARM`		
`RECORD_AUDIO`		
`MANAGE_EXTERNAL_STORAGE`		

B2. Grupos y nombres

a) ¿Qué es un **grupo de permisos**? Si la app pide `ACCESS_FINE_LOCATION` y `ACCESS_COARSE_LOCATION` juntos, ¿cuántos diálogos ve la persona?

b) ¿Qué prefijo lleva el nombre completo de un permiso? Escribí el nombre completo del permiso de cámara.

c) ¿Qué son los permisos de firma (signature)? ¿Por qué no se usan en apps comunes?

B3. Leé el manifiesto

Este es el `AndroidManifest.xml` que generó `npx expo prebuild`. Observalo y respondé:

```
android/app/src/main/AndroidManifest.xml (generado)
```

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
          xmlns:tools="http://schemas.android.com/tools">
  <uses-permission android:name="android.permission.INTERNET" />
  <uses-permission android:name="android.permission.CAMERA" />
  <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
  <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
  <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"
                   android:maxSdkVersion="32" />
  <uses-permission android:name="android.permission.RECORD_AUDIO"
                   tools:node="remove" />
</manifest>
```

a) ¿Qué permisos se conceden solos al instalar y cuáles hay que pedir además en tiempo de ejecución?

b) ¿Qué significa `android:maxSdkVersion="32"`? ¿Qué permisos reemplazan a `READ_EXTERNAL_STORAGE` desde Android 13?

c) ¿Qué hace `tools:node="remove"`? Escribí el fragmento de `app.json` que lo genera.

d) Necesitás agregar `SCHEDULE_EXACT_ALARM`, que ninguna librería declara. ¿En qué propiedad de `app.json` va?

e) ¿Por qué en Expo no conviene editar este archivo a mano? ¿Funciona `blockedPermissions` en Expo Go?

B4. Así pregunta Android

a) ¿Se puede poner texto propio dentro del diálogo de Android? Entonces, ¿dónde tenés que explicar para qué usás el permiso?

b) ¿Qué diferencia hay entre **“Mientras la app está en uso”** y **“Solo esta vez”**? ¿Para qué permisos existe esta última opción?

c) En Android 11+, ¿qué pasa si la persona toca “No permitir” dos veces? ¿Qué valor toma `canAskAgain`?

d) La ubicación aproximada tiene una precisión de unos 3 km². Dá un ejemplo de función que anda bien con ella y otro de una que no.

e) Si la persona revoca un permiso desde Ajustes mientras la app está abierta, ¿qué hace Android con el proceso de la app?

B5. Android, versión por versión

Completá en qué versión apareció cada cambio y qué implica para tu código.

Cambio	Versión (API)	Qué implica para el código
Nacen los permisos en tiempo de ejecución		
La ubicación en segundo plano pasa a ser un permiso aparte		
Dos rechazos equivalen a “no volver a preguntar”		
Ubicación precisa o aproximada		

Cambio	Versión (API)	Qué implica para el código
Las notificaciones pasan a ser un permiso (`POST_NOTIFICATIONS`)		
Acceso parcial a fotos y videos (“Seleccionar fotos”)		
Permiso de red local (`ACCESS_LOCAL_NETWORK`)		

¿Qué es el `targetSdkVersion`? ¿A qué API apunta Expo SDK 57 y qué exige Google Play desde el 31 de agosto de 2026?

Parte C · iOS: textos de propósito y acceso parcial

C1. Claves del Info.plist

Escribí la clave que corresponde a cada recurso. Si no lleva clave, indicalo.

Recurso	Clave en Info.plist
Cámara	
Micrófono	
Ubicación en uso	
Ubicación siempre	
Fotos (leer)	
Fotos (solo guardar)	
Contactos	
Face ID	
Rastreo entre apps (ATT)	
Notificaciones	

C2. Sin texto, no hay permiso

a) ¿Qué pasa si falta el texto de propósito y la app intenta usar la cámara? ¿Y al subir la build a App Store Connect? Mencioná el código de error.

b) ¿Cuántas veces muestra iOS el diálogo de un permiso? Si la persona dice que no, ¿qué le queda a la app?

c) En el diálogo de iOS, ¿qué parte arma el sistema y cuál escribís vos? En Expo, ¿dónde se configura ese texto?

d) Una librería usa una API protegida que tu app nunca llama y Apple rechaza la build. ¿Por qué pasa y cómo se resuelve?

C3. Probador de textos de propósito

Evaluá cada texto para `NSLocationWhenInUseUsageDescription` según la pauta 5.1.1 de Apple. Indicá si lo aprobarías y reescribí los que no, con el molde **“Usamos tu [dato] para [beneficio concreto]”**.

Texto propuesto	¿Aprobado?	Versión mejorada
“Necesitamos tu ubicación.”		
“Para mejorar tu experiencia.”		
“Usamos tu ubicación para mostrarte las farmacias de turno más cercanas.”		
“La app no funciona sin ubicación.”		
“Ubicación requerida por el sistema.”		

C4. Acceso parcial y gradual

Para cada caso indicá desde qué versión de iOS existe y qué valor o función de Expo lo detecta o lo pide.

Caso	iOS	En Expo
Fotos limitadas (“Seleccionar fotos”)		
Ubicación aproximada (interruptor “Precisa”)		
“Permitir una vez” (ubicación)		
Ubicación escalonada: de “al usar la app” a “Siempre”		
Notificaciones provisionales		

Caso	iOS	En Expo
Rastreo entre apps (App Tracking Transparency)		

¿Qué versión mínima de iOS requiere Expo SDK 57? ¿Qué consecuencia práctica tiene eso sobre la tabla anterior?

C5. Android vs iOS, lado a lado

Completá la tabla comparativa.

	Android	iOS
¿Dónde se declara?		
Texto en el diálogo		
Oportunidades para preguntar		
“Solo esta vez” / “Permitir una vez”		
Notificaciones		
Ubicación en segundo plano		
Si falta la declaración		

Parte D · Expo: una API, dos plataformas

D1. Módulos y funciones

Indicá qué módulo de Expo y qué hook o función de permiso usarías en cada caso.

Necesidad	Módulo	Hook / función de permiso
Escanear un QR con la cámara		
Grabar video con sonido		
Mostrar farmacias cercanas		
Seguir un envío con la app en segundo plano		
Mostrar un recordatorio		
Grabar una nota de voz		

Necesidad	Módulo	Hook / función de permiso
Galería propia dentro de la app		
Publicidad personalizada entre apps (iOS)		

Un tutorial usa `import * as Permissions from 'expo-permissions'`. ¿Qué te dice eso sobre el tutorial?

D2. Consultar, pedir y el hook

```
el patrón, con expo-location
import * as Location from 'expo-location';

const actual = await Location.getForegroundPermissionsAsync();
const respuesta = await Location.requestForegroundPermissionsAsync();

const [permiso, pedirPermiso, consultarPermiso] =
  Location.useForegroundPermissions();
```

a) ¿Qué diferencia hay entre `get…PermissionsAsync()` y `request…PermissionsAsync()`? ¿Cuál puede mostrar un diálogo?

b) El hook devuelve tres elementos. ¿Qué es cada uno? ¿Qué vale `permiso` en el primer render y qué tiene que mostrar la pantalla mientras tanto?

c) Si `canAskAgain` es `false`, ¿qué pasa al llamar a `request…()`? ¿Qué problema aparece si se lo llama en un bucle?

D3. Leé la respuesta

Para cada `PermissionResponse`, indicá qué debería hacer la app.

Respuesta	Qué hace la app
`null`	
`{ status: 'granted', granted: true }`	
`{ status: 'undetermined', canAskAgain: true }`	
`{ status: 'denied', canAskAgain: true }`	
`{ status: 'denied', canAskAgain: false }`	
`{ status: 'granted', accessPrivileges: 'limited' }`	
`{ status: 'granted', android: { accuracy: 'coarse' } }`	

¿Cuáles son los **dos campos** que deciden casi todo? ¿Qué pregunta responde cada uno?

D4. Simulación del ciclo completo

Android 14, app recién instalada, `expo-location`. Escribí el estado después de cada paso.

#	Acción	status	canAskAgain	¿Diálogo?
1	`getForegroundPermissionsAsync()`			
2	`requestForegroundPermissionsAsync()` → "No permitir"			
3	`requestForegroundPermissionsAsync()` → "No permitir"			
4	`requestForegroundPermissionsAsync()`			
5	`Linking.openSettings()` → habilita "Mientras la app está en uso" → vuelve y la app consulta			
6	Igual que el 5, pero la app **no** vuelve a consultar: ¿qué muestra?			

Repetí los pasos 1 a 4 en **iOS**. ¿En qué paso cambia el resultado y por qué? ¿Qué pasa en iOS si la persona elige "Permitir una vez" y después cierra y reabre la app?

D5. Configurar app.json

Escribí el bloque `expo.plugins` (y lo que haga falta en `expo.android`) para una app que:

- lee códigos QR con la cámara pero **no graba video** (sin micrófono en iOS y sin `RECORD_AUDIO` en Android);
- usa la ubicación **solo en primer plano** para mostrar puntos de reciclaje cercanos;
- no debe tener `SYSTEM_ALERT_WINDOW` en el manifiesto final.

¿Por qué `expo-camera` agrega `RECORD_AUDIO` en Android aunque la app no grabe sonido?

D6. Expo Go no es tu app

a) ¿Por qué tus textos de propósito no aparecen cuando probás en Expo Go?

b) ¿Qué se puede probar en Expo Go y qué necesita una development build? Pensá en push en Android y en ubicación en segundo plano.

c) Escribí los comandos para ver los archivos nativos generados y para compilar una development build en Android.

d) ¿Cómo volvés un permiso a "sin decidir" sin reinstalar, en Android (`adb`) y en el simulador de iOS (`xcrun simctl`)?

Parte E · Recetas: leer y corregir código

E1. Encontrá los errores (cámara)

Esta pantalla funciona la primera vez, pero tiene varios problemas. Encontrá **al menos tres** y escribí la versión corregida con los cuatro estados (cargando → concedido → preguntable → bloqueado).

`src/app/escanear.tsx`

```
import { CameraView, useCameraPermissions } from 'expo-camera';
import { Button } from 'react-native';

export default function Escanear() {
  const [permiso, pedirPermiso] = useCameraPermissions();

  if (permiso.status === 'granted') {
    return <CameraView style={{ flex: 1 }} facing="back" />;
  }

  return (
    <Explicacion texto="Usamos la cámara para leer el QR del aula.">
      <Button title="Permitir" onPress={pedirPermiso} />
    </Explicacion>
  );
}
```

E2. El orden importa (ubicación)

`src/lib/ubicacion.ts`

```
export async function activarSeguimiento() {
  const siempre = await Location.requestBackgroundPermissionsAsync();
  if (!siempre.granted) return false;

  const enUso = await Location.requestForegroundPermissionsAsync();
  return enUso.granted;
}
```

a) ¿Qué está mal? ¿Qué ocurre en iOS y qué ocurre en Android 11+?

b) Reescribí la función correctamente.

c) Escribí `obtenerUbicacion()` para que devuelva las coordenadas y un booleano `aproximada` que contemple Android (`accuracy`) e iOS (`accuracy`). Si se deniega, debe devolver `null` sin romper la app.

d) ¿Hace falta segundo plano para “farmacias cercanas”? ¿Qué exige Google Play si lo pedís?

E3. Notificaciones mudas

Un compañero dice: “En Android 14 llamo a `requestPermissionsAsync()` y nunca aparece el diálogo”.

a) ¿Cuál es la causa más probable? ¿Qué hay que hacer antes de pedir el permiso?

b) ¿Qué pasa con este permiso en Android 12 o anterior?

c) ¿Qué opción de iOS permite recibir notificaciones sin diálogo? ¿Cómo detectás en código ese estado?

d) ¿En qué momento conviene pedir este permiso? Dá un ejemplo de buen y de mal momento.

E4. Fotos sin pedir permiso

Necesidad	Herramienta	Permiso
Elegir una foto de perfil		
Sacar una foto en el momento		
Editor que muestra toda la galería		
Adjuntar la foto de un comprobante		

¿Por qué Google Play restringe `READ_MEDIA_IMAGES` y `READ_MEDIA_VIDEO`? ¿Desde cuándo? ¿Qué es "el mejor permiso"?

E5. Volver de Ajustes

Completá los espacios del hook y respondé.

`src/hooks/useRevisarAlVolver.ts`

```
import { useEffect, useRef } from 'react';
import { AppState } from 'react-native';

export function useRevisarAlVolver(revisar: () => void) {
  const anterior = useRef(AppState.currentState);

  useEffect(() => {
    const sub = AppState.addEventListener('change', (nuevo) => {
      const estabaAfuera = _____________________;
      if (estabaAfuera && _____________________) revisar();
      anterior.current = nuevo;
    });
    return () => _____________________;
  }, [revisar]);
}
```

a) ¿Por qué los hooks de Expo no se enteran solos de que el permiso cambió en Ajustes?

b) ¿Qué función del hook de cámara le pasarías a `useRevisarAlVolver`?

c) ¿Por qué no conviene guardar el estado del permiso en un store global "para siempre"?

Parte F · Buenas prácticas y tiendas

F1. Pedir en contexto

Indicá el momento justo para pedir cada permiso y la alternativa si la persona dice que no.

App y permiso	¿Cuándo pedirlo?	Alternativa si lo deniegan
Delivery · ubicación		
Billetera virtual · cámara		

App y permiso	¿Cuándo pedirlo?	Alternativa si lo deniegan
Red social · contactos		
Mensajería · micrófono		
Turnos médicos · notificaciones		
Editor de fotos · galería completa		

¿Por qué pedir cuatro permisos apenas se abre la app “entrena el No permitir automático”?

F2. La pantalla previa

a) ¿Qué tres elementos tiene una buena pantalla previa al diálogo del sistema?

b) ¿Por qué el botón debe decir “Continuar” y no “Permitir”?

c) ¿Qué dice la pauta 5.1.1(iv) de Apple sobre botones como “Ahora no” o “Cerrar” en esa pantalla?

d) ¿Cómo cambia la pantalla si el permiso ya está bloqueado?

F3. Mínimo privilegio

Para cada caso elegí la opción más chica que alcanza y justificá.

a) Ubicación precisa o aproximada para “puntos de reciclaje de tu barrio”.

b) Ubicación “en uso” o “siempre” para “registrar una carrera con el celular en el bolsillo”.

c) Selector del sistema o permiso de galería para “subir la foto del DNI”.

d) `expo-camera` con o sin micrófono para “escanear el QR del aula”.

F4. Tiendas

a) Nombrá dos requisitos de **Google Play** y dos de **App Store** relacionados con permisos y privacidad.

b) ¿Qué es el manifiesto de privacidad (`PrivacyInfo.xcprivacy`) y dónde se configura en Expo?

c) ¿Qué suma Android 17 y qué va a exigir Google Play desde el 27/01/2027 a las apps que hoy piden `READ_CONTACTS`?

F5. Errores comunes

Para cada situación explicá la causa y la solución:

a) La consola de Xcode muestra: “This app has crashed because it attempted to access privacy-sensitive data without a usage description.”

b) “En Expo Go anda, en la build no”: en la development build la cámara se deniega sin mostrar ningún diálogo.

c) Después de habilitar la ubicación en Ajustes, la app sigue diciendo “bloqueado”.

d) La pantalla queda en “Pidiendo permiso…” para siempre porque el código llama a `request…()` cada vez que se renderiza.

e) Una app de turnos no permite entrar a ninguna pantalla si se deniegan las notificaciones.

Parte G · Desarrollo del sistema: “Mantenimiento IPF”

El Instituto quiere una app para que alumnos y docentes **reporten desperfectos del edificio** (una luz quemada, una canilla que pierde, un proyector roto) y reciban un aviso cuando se resuelven. Vas a desarrollarla con **Expo Router (SDK 57)** aplicando todo lo visto. La app usa **cámara**, **ubicación** y **notificaciones**, y elige fotos **sin pedir permiso de galería**.

G1. Rutas requeridas

La app debe responder exactamente a estas URLs. **Parte del trabajo es diseñar el árbol de carpetas** que las genera (incluilo en el README).

URL	Pantalla	Permiso / detalle
`/`	Inicio: lista de reportes propios con su estado (pendiente, en curso, resuelto) y botón “Nuevo reporte”.	Ninguno. Tab “Reportes”.
`/reporte/nuevo`	Formulario: título, descripción, aula y categoría (electricidad, agua, mobiliario, equipos). Botones para escanear, adjuntar foto y marcar ubicación.	Ninguno al abrir: cada permiso se pide recién al tocar su función.
`/reporte/escanear`	Lee el QR pegado en la puerta del aula y completa el campo “aula”.	Cámara (`useCameraPermissions`), con los cuatro estados.
`/reporte/foto`	Adjuntar foto del desperfecto: “Sacar foto” o “Elegir de la galería”.	Cámara para sacar; selector del sistema (sin permiso) para elegir.
`/reporte/ubicacion`	Marca dónde está el problema (patio, estacionamiento, pasillo) con la ubicación actual.	Ubicación en primer plano. Si se deniega: elegir el sector a mano.
`/reporte/[id]`	Detalle del reporte: datos, foto, ubicación o sector y estado. Botón de prueba “Simular resolución”.	Validar el `id`; mensaje si no existe.
`/avisos`	Bandeja de avisos dentro de la app.	Alternativa a las notificaciones. Tab “Avisos”.
`/permisos`	Panel con el estado de cada permiso (concedido, preguntable, bloqueado, parcial) y botón para pedir o abrir Ajustes.	Tab “Permisos”. Se actualiza al volver de Ajustes.
`/permisos/[recurso]`	Pantalla previa: beneficio en una frase, botón “Continuar” y aclaración de que se puede cambiar en Ajustes.	`presentation: "modal"`.
`/ajustes-app`	Interruptor “Avisarme cuando se resuelva mi reporte”.	Notificaciones, pedidas al activar el interruptor.

G2. Requisitos funcionales

1. **Hook unificado.** En `src/hooks/usePermiso.ts` implementá `usePermiso(recurso)` que devuelva `'cargando' | 'concedido' | 'preguntable' | 'bloqueado'`, un indicador `parcial` (fotos limitadas o ubicación aproximada) y las funciones `pedir()` y `abrirAjustes()`. Debe actualizarse al volver de Ajustes usando `AppState`.
2. **Escáner con cuatro estados.** `/reporte/escanear` muestra el indicador de carga, la cámara, la explicación con “Continuar” o el aviso de bloqueado con “Abrir Ajustes”, según

corresponda. Si la cámara se deniega, se puede escribir el código del aula a mano.

3. **Foto sin permiso de galería.** “Elegir de la galería” usa `ImagePicker.launchImageLibraryAsync`; la app no declara `READ_MEDIA_IMAGES` ni pide acceso a la galería.
4. **Ubicación respetuosa.** Pedí solo primer plano. Si es aproximada, no muestres coordenadas exactas sino “Ubicación aproximada”; si se deniega, el reporte se carga con el sector elegido a mano.
5. **Notificaciones en contexto.** El permiso se pide únicamente al activar el interruptor de `/ajustes-app`. En Android creá el canal `"reportes"` antes de pedir. “Simular resolución” programa una notificación local a los 10 segundos; si el permiso está denegado, el aviso aparece en `/avisos`.
6. **Pantallas previas.** Antes de cada diálogo del sistema se muestra `/permisos/[recurso]` con un único botón “Continuar”, sin “Ahora no” ni “Cerrar”. Si el permiso está bloqueado, explica cómo habilitarlo y el botón pasa a ser “Abrir Ajustes”.
7. **La app nunca se bloquea.** Ningún permiso denegado impide crear un reporte: cada función tiene su alternativa (código a mano, foto desde el selector, sector a mano, bandeja de avisos).
8. **No insistir.** Si la persona ya dijo que no, la pantalla previa no vuelve a aparecer sola: solo cuando toca otra vez la función.
9. **Panel de permisos.** `/permisos` muestra cada recurso con su estado, la plataforma (`Platform.OS`) y el matiz cuando corresponda (`accessPrivileges`, `accuracy`).

G3. Requisitos técnicos

- Proyecto creado con `npx create-expo-app@latest`, Expo SDK 57, TypeScript y Expo Router. Paquetes instalados solo con `npx expo install` (`expo-camera`, `expo-location`, `expo-image-picker`, `expo-notifications`). Prohibido usar `expo-permissions`.
- `app.json` con el plugin de cada módulo y textos de propósito propios que cumplan el molde “Usamos tu [dato] para [beneficio concreto]”.
- Mínimo privilegio: `microphonePermission: false` y `recordAudioAndroid: false` en `expo-camera`; `blockedPermissions` con los permisos que la app no usa (justificá cada uno en el README).
- Organización: rutas en `src/app`, componentes en `src/components`, hooks en `src/hooks`, lógica de permisos en `src/lib`.
- Probado en Expo Go y en al menos una development build (Android o iOS).

G4. Desafíos opcionales (suman puntos extra)

- **Auditoría del manifiesto:** correr `npx expo prebuild --clean` y listar en el README cada permiso del `AndroidManifest.xml` generado, quién lo agrega y si lo quitaste.
- **Notificaciones provisionales:** en iOS, pedir con `allowProvisional: true` y mostrar en `/permisos` cuando el estado es provisional.
- **Nota de voz:** grabar una descripción con `expo-audio` (`requestRecordingPermissionsAsync`), activando el micrófono solo para esta función.
- **Bloqueo por plataforma:** pantalla de “permiso bloqueado” con el camino exacto en Ajustes para Android y para iOS.

G5. Entregables

1. Repositorio con el proyecto funcionando (sin `node_modules`).
2. `RESPUESTAS.md` con las respuestas de las Partes A a F.
3. `README.md` con: árbol de carpetas de `src/app`, el bloque `plugins` de `app.json` comentado, una tabla de permisos (recurso, módulo, cuándo se pide, alternativa) y capturas (o un video corto) de: escáner en sus cuatro estados, ubicación concedida y denegada, foto desde el selector, notificación recibida y panel de permisos antes y después de volver de Ajustes.
4. Una captura del diálogo del sistema en una development build donde se vea tu texto de propósito.

G6. Preguntas para la defensa oral

En la defensa del trabajo se te pedirá explicar, sobre tu propio código:

1. Mostrá el código que decide entre “Continuar” y “Abrir Ajustes”. ¿Qué campo de la respuesta lo decide?
2. Revocá la cámara desde Ajustes con la app abierta y volvé. ¿Qué hace Android con la app? ¿Cómo se entera tu código del cambio?
3. ¿Por qué tu app no pide permiso para elegir una foto, pero sí para sacarla?
4. ¿Qué cambia en la app si la persona elige ubicación aproximada? ¿Y si la deniega?
5. ¿Por qué tus textos de propósito no aparecían en Expo Go?
6. Si mañana agregás la nota de voz, ¿alcanza con publicar una actualización OTA? ¿Por qué?

G7. Criterios de evaluación

Criterio	Puntos
Partes A a F (preguntas y ejercicios)	40
Hook `usePermiso` y manejo de los cuatro estados, incluido volver de Ajustes	15
Pedidos en contexto, pantallas previas y alternativas cuando se deniega	15
Configuración de `app.json`: plugins, textos de propósito y mínimo privilegio	10
Funciones del sistema: escáner, foto, ubicación y notificaciones	10
Organización del código, README y defensa oral	10
Total	**100**
Desafíos opcionales	hasta +10